Queue overflow

Queue Operations
1. Enque (insert) 2. Deque (delete) 3. Display 4. Exit
Enter your choice (1-4): 2
deleted element is 1

Queue operations
1. Enque(insert) 2. Deque(delete) 3. Display 4. Exit
Enter your choice (1-4): 2
deleted element is 2.

Queue Operations
1. Enque (insert) 2. Deque (delete) 3. Display 4. Exit
Enter your choice (1-4): 3
Elements of queue are:
3
4
5

Queue operations
1. Enque (insert) 2. Deque (delete) 3. Display 4. Exit
Enter your choice (1-4): 4
Exit...

13/10/25

3/11/25

WAP to simulate the working of a circular queue of integers using an array. Provide the following operations insert, Delete & display. the program should print appropriate messages for queues empty and queue overflow conditions.

~~Algorithm~~ Pseudo code:

1. Declare an array of size N. and an integer x, to insert.
2. Define N globally, Front = rear = -1.
3. TO insert
   check if (front == -1 and Rear == -1) print underflow
   set Front and rear to 0 and insert x at rear.
   check if rear + 1 % N == front print overflow
   Otherwise set rear = rear + 1 % N
   insert x at rear position.

4. To delete

check if front and rear both equals to -1 print underflow

check if front and rear are equal

print the element at front and set front and rear to -1.

otherwise print the element at front and set front to (front+1) % N.

5. To display

check if front and rear both equals to -1 print underflow

otherwise use a for loop which starts at front and under condition i<= rear

updation i = (i+1) % N.

print the element at front.

6. End.

3/4/25

```
#include <stdio.h>
# define N 5
int front = -1;
int rear = -1;
int queue[N];
void enqueue (int x){
    if ((front==0 && rear==N-1) || ((rear+1) % N == front)){
        printf("queue is full \n");
    }
    else if (front==-1 && rear==-1 ){
        front=rear = 0;
        queue[rear]=x;
    }
    else {
        rear= (rear+1)%N;
        queue[rear] =x;
    }
}
```

```
void enqueue ( ){
    if( front == -1 && rear == -1)
      {
       printf ("Queue is empty \n No element to delete");
      }
     else if (front == rear){
          printf ("The deleted element is %d \n", queue[front]);
          front = rear = -1;
     }
     else {
     printf (" The deleted element is %d \n", queue[front]);
     front = (front+1) % N;
      }
}
void display ( ){
    if (front == -1 && rear == -1) {
       printf (" Queue is empty \n ");
     }
    else {
       printf ( " The queue elements are : \n ");
       int i;
       for (i=front ; i != rear ; i= (i+1) % N ){
            printf ("%d ", queue[i]);
            printf ("\n ");
        }
       printf ("%d", queue[rear]);
     }
  }
int main ( ) {
  int ch, x;
  do {
   printf (" Enter your choice \n 1. enqueue \n 2. dequeue \n 3. display \n
       4. exit : ");
   scanf ("%d", &ch);
   switch (ch){
case 1 :
    Printf("Enter element to enter: ");
    scanf("%d", &x);
    enqueue(x);
```

```
        break;
    case 2:
        dequeue();
        break;
    case 3:
        display();
        break;
    case 4:
        printf("exit.");
        break;
    default:
        printf("Choice out of range");
    } while(ch!=4);
    return 0;
}
```

O/P:

```
Enter your choice
1. enqueue
2. dequeue
3. display
4. exit : 2
Queue is empty
No element to delete
Enter your choice
1. enqueue
2. dequeue
3. display
4. exit : 1
Enter element to enter : 1
Enter your choice
1. enqueue
2. dequeue
3. display
4. exit : 2
The deleted element is 1
```

Enter your choice

1. enqueue

2. dequeue

3. display

4. exit : 1

Enter element to enter : 1

Enter your choice

1. enqueue

2. dequeue

3. display

4. exit

Enter element to enter : 2

Enter your choice

1. enqueue

2. dequeue

3. display

4. exit : 3

The queue elements are :

1

2

Enter your choice

1. enqueue

2. dequeue

3. display

4. exit : 4

exit...

3/11/25

O/P Seen